Design and Analysis of Algorithms

Question Bank 2025-26

UNIT-1 Problem solving and Algorithmic Analysis

CO1: Analyze the efficiency of algorithms using mathematical techniques to evaluate algorithm performance.

Basic Questions: (Blooms Level- Understanding)

1. Analyze the importance of algorithms in solving computational problems and their impact on efficiency.
2. Differentiate between best-case, worst-case, and average-case complexities with suitable examples.
3. Analyze asymptotic notations and explain any two notations in terms of algorithm growth.
4. Disucss the concept of recurrence relations and determine their role in analyzing recursive algorithms.
5. Analyze the significance of time complexity in algorithm design and performance evaluation.
6. Classify different types of time complexities and compare their efficiency.
7. Analyze and apply the Master Theorem to solve recurrence relations in algorithm analysis.
8. Compare deterministic and non-deterministic algorithms with examples based on their performance.
9. Explain the relationship between P, NP, NP-Hard, and NP-Complete using a Venn diagram.

Moderate Questions: (Blooms Level- Applying)

1. Analyze the relationship between upper bound and lower bound in evaluating algorithm efficiency.
2. Classify asymptotic notations based on their functions in algorithm analysis.
3. Analyze the best-case, worst-case, and average-case time complexities of Linear Search.
4. Analyze the best-case, worst-case, and average-case time complexities of operations in a Binary Search Tree.
5. Explain the importance of recurrence relations in algorithm analysis using a suitable example

6. Analyze how the Master Theorem is applied to solve recurrence relations efficiently.
7. Analyze and solve the recurrence relation using the Master Theorem: T(n) = 8T(n/2) + n^2
8. Formulate the recurrence relation of Binary Search and solve it using the Master Theorem.
9. Analyze and determine the upper bound, lower bound, and average bound for the function:

f(n) = 2n^2 + 3n + 4

11. Formulate the recurrence relation for Merge Sort.

12. Analyze and determine the upper bound for the function:

f(n) = n^2 \log n + n

12. Compute and justify the upper bound for the function:

f(n) = 2n^3 + 4n + 5

13. Analyze and compute the time complexity of Merge Sort and explain its efficiency.
14. Solve the recurrence T(n)=2T(n/2)+n and interpret its time complexity.
15. Solve the recurrence T(n)=4T(n/2)+n^ using the Master Theorem.
16. Determine the time complexity of T(n)=T(n/2)+1and relate it to a standard algorithm.
17. Analyze whether the Master Theorem is applicable to T(n)=2T(n/2)+nlogf_0n and justify your answer.

Descriptive/HOTS Questions: (Blooms Level- Analyzing)

1. Analyze the application of asymptotic notations in evaluating algorithm efficiency, and interpret their impact using multiple examples.
2. Investigate the formulation of recurrence relations and apply the Master Theorem to derive their time complexity, supporting your answer with a detailed example.
3. Formulate the recurrence relation for multiplication of large digit number and derive its time complexity using the Master Theorem, justifying each step.
4. Compare and evaluate the time complexities of Merge Sort and Quick Sort across best, worst, and average cases, and infer their suitability for different scenarios.
5. Critically evaluate a real-world scenario where best-case analysis is more significant than worst-case analysis, and justify your reasoning based on algorithm performance.

UNIT 2- Divide and Conquer Strategy, Greedy Strategy

CO2: Analyze the efficiency of algorithm using either Divide and Conquer or Greedy strategy

Basic Questions (Bloom's Level: Understanding)

1. How does binary search utilize the divide and conquer strategy?
2. Differentiate between quick sort and merge sort in terms of efficiency.
3. Explain how the divide and conquer strategy applies to multiplication of large digit number.
4. Write the control abstraction of the greedy strategy and analyse its time complexity.
5. Discuss the knapsack problem, and how is it solved using the greedy method?
6. How does Dijkstra's algorithm find the shortest path in a graph?

Moderate Questions (Bloom's Level: Applying)

1. Apply the quick sort algorithm to the following array: [7, 2, 9, 4, 3, 1, 5], and show the steps.
2. Analyze how the greedy approach optimizes the job scheduling problem. Discuss with example.
3. Use the divide and conquer approach to solve the maximum sub-array problem for the array [−2, 1, −3, 4, −1, 2, 1, −5, 4].
4. Analyze the time complexity of merge sort using the divide and conquer approach.
5. Compare the time complexities of quick sort and merge sort in their best, worst, and average cases.
7. Apply divide and conquer to solve the integer multiplication problem using Karatsuba's algorithm.
8. Solve the fractional knapsack problem for the given weights and values:
   weight: (2,3,5,7,1,4,1) Profits: (10,5,15,7,6,18,3) m=15.
9. Apply Dijkstra's algorithm to the following graph: A→B(2), A→C(4), B→C(1), B→D(7), C→D(3). Find the shortest path from node A to all other nodes.
6. Compare the greedy approach with dynamic programming for solving the 0/1 knapsack problem.
7. Apply merge sort to the following array [12,11,13,5,6,13,86.41,10,7] and show the process step-by-step.

8. Analyze how the divide and conquer strategy is used to solve the integer multiplication problem. Compare its time complexity with brute force methods.

9. Apply binary search to find the position of the element 9 in the sorted array [2, 3, 5, 7, 9, 11, 13, 15]. Write Binary Search Algorithm with time complexity.

10. Analyze the limitations of the greedy algorithm in solving the fractional knapsack problem when items cannot be split.

11. Use the greedy algorithm to solve the job scheduling problem for jobs with the following deadlines and profits:

N=4, Profits:(30, 35, 20, 25) Deadline:(2, 1, 2, 1)

Analyze the time complexity of Dijkstra's algorithm. Why is it more efficient than Bellman-Ford for graphs with non-negative weights?

12. Compare quick sort and merge sort in terms of space complexity and efficiency for different types of input data (random, nearly sorted, etc.).

Higher Order Thinking Questions (HOTS) (Bloom's Level: Analyze, Evaluating, Creating)

1. Evaluate the limitations of quicksort and propose an optimization to improve its worst-case performance.
2. Elaborate the practical use of divide and conquer in integer arithmetic operations. Would a different approach be better for large-scale multiplication?
3. Evaluate the performance of the divide and conquer strategy in solving recursive problems like matrix multiplication and compare it with dynamic programming.
4. Evaluate a situation where the greedy approach fails to provide the optimal solution for the knapsack problem. How can dynamic programming solve it optimally?
5. Propose a variant of Dijkstra's algorithm that could work efficiently with negative edge weights.
6. Assess the limitations of the greedy strategy in solving the job scheduling problem and propose an alternative solution.
7. Create a modification of the greedy algorithm to solve the traveling salesman problem. Explain why this modification may or may not work.
8. Evaluate the differences between the greedy and divide and conquer strategies. Which is more suitable for real-world optimization problems?

UNIT 3- Dynamic Programming Strategy

CO3: Apply computational techniques to solve optimization and decision-making problems using either Dynamic programming or Branch and Bound.

Basic Questions (Bloom's Level: Understanding)

1. Explain the principle of optimality in dynamic programming.
2. What are the binomial coefficients, and how are they computed using dynamic programming?
3. Write the dynamic programming recurrence relation for computing binomial coefficients.
4. What is the 0/1 knapsack problem? How does dynamic programming approach it differently from the greedy method?
5. Explain the Floyd-Warshall algorithm for finding the all-pairs shortest path in a graph.
6. What is the significance of the binomial coefficient in combinatory and probability theory?
7. What is a binomial coefficient? How is it defined mathematically?
8. Describe the sum of subset problem in dynamic programming.
9. How does dynamic programming reduce the time complexity of recursive algorithms?
10. Discuss the principle of Dynamic Programming and write Binomial Coefficient formula.

Moderate Questions (Bloom's Level: Applying)

1. Solve the following instance of the 0/1 knapsack problem using dynamic programming: Items = [weight: 2,value: 3,weight: 3,value: 4,weight: 4,value: 5, weight: 2,value: 3,weight: 3,value: 4,weight: 4,value: 5], Knapsack capacity = 5.
2. Use dynamic programming to solve the sum of subset problem for the set [3,34,4,12,5,2] and a target sum of 9.
3. Apply the Floyd-Warshall algorithm to find the shortest paths for the following graph:

   A→B(8), B→C(1), A→D(1), D→C(9), D→B(2), C→A(4)
4. Analyze the time complexity of the Bellman-Ford algorithm. How does it compare to Dijkstra's algorithm?
5. Apply dynamic programming solve the multistage graph problem for following graph :
   (Any Graph with values)
6. Analyze the difference between the greedy solution and the dynamic programming solution for the 0/1 knapsack problem.

7. Explain how to fill the table (array) in the dynamic programming solution for binomial coefficients. What values are filled in the base cases?
8. Use dynamic programming to compute the binomial coefficient for C(6,3), and compare it to the recursive approach.
9. Compute C(10, 4) using dynamic programming. Demonstrate how the binomial coefficient can be computed using both full DP table and an optimized solution.
10. Explain the role of memoization in dynamic programming. How does it improve performance over simple recursion?
11. Solve the following 0/1 knapsack problem using dynamic programming: Items = [weight: 2,value: 3,weight: 3,value: 4,weight: 4,value: 5,weight: 5,value: 8], Knapsack capacity = 5.
12. Use appropriate strategy to compute the sum of subset problem for the set [2,3,5,7,10] with a target sum of 14.
13. Analyze how dynamic programming improves the time complexity of solving the binomial coefficient problem compared to recursive methods.
14. Use dynamic programming to compute the cost of constructing an Optimal Binary Search Tree (OBST) for the following keys and frequencies: Keys = A,B,C,D, Frequencies = 10,20,30,40.
15. Apply the Bellman-Ford algorithm to detect negative weight cycles in the following graph: A→B(3),B→C(2),C→A(−6),A→D(5).
16. Analyze the difference between the Floyd-Warshall algorithm and Dijkstra's algorithm for solving the shortest path problem in weighted graphs.
17. Solve the multistage graph problem using dynamic programming for the following graph with stages and weights: S1→S2(2),S1→S3(4),S2→S4(3),S3→S4(5)
18. Analyze the role of overlapping subproblems in dynamic programming, using the example of the Fibonacci sequence. Why is dynamic programming more efficient than recursion for such problems?
19. Apply the dynamic programming approach to solve given binomial coefficients values, N=5, K=3.

Higher Order Thinking Skills (HOTS) Questions (Bloom's Level: Analyzing, Evaluating, Creating)

1. Evaluate the benefits and limitations of dynamic programming in solving the all-pairs shortest path problem using the Floyd-Warshall algorithm.
2. Create a dynamic programming solution for a new variant of the knapsack problem where items can be divided into multiple groups, each with its own constraints.
3. Evaluate the effectiveness of dynamic programming in solving the OBST problem for large datasets. How could you optimize the space complexity?
4. Design a dynamic programming algorithm for solving the traveling salesman problem. Compare it with the greedy approach in terms of efficiency and accuracy.
5. Propose an alternative approach to solving the sum of subset problem when the set contains negative numbers. How would you modify the dynamic programming solution?
6. Evaluate the efficiency of dynamic programming for multistage graph problems compared to greedy and backtracking approaches.
7. Analyze a situation where dynamic programming would not be the best approach to solve the 0/1 knapsack problem, and propose an alternative solution.
8. Propose a dynamic programming algorithm to solve a multistage decision problem in a business optimization scenario. How would you model the stages and decisions?
9. Form binomial coefficient formula and Compute C(8, 8) using dynamic programming.
10. Consider two sequences: X = "ABCBDAB" Y = "BDCABA" What is the length of the Longest Common Subsequence? Use Dynamic programming approach.
11. Write the recurrence relation for Fibonacci using Dynamic Programming.
12. Explain why the 0/1 Knapsack problem cannot be solved using a greedy approach but can be solved using Dynamic Programming

Unit 4 : Branch and Bound & Backtracking Strategies

Branch and Bound

CO3: Apply computational techniques to solve optimization and decision-making problems using either Dynamic programming or Branch and Bound.

Basic Questions (Bloom's Level: Understanding)

1. Explain the basic concept of the Branch and Bound technique, and the difference from other algorithm design strategies like Greedy and Dynamic Programming.
2. Describe how Branch and Bound is applied to solve the 0/1 Knapsack problem. Explain the role of bounding functions and how they improve the efficiency of the algorithm.
3. Explain the difference between depth-first search (DFS) and breadth-first search (BFS) strategies in the context of Branch and Bound. When would you prefer one over the other?
4. Explain how the FIFO queue is used to explore the solution space in the Branch and Bound algorithm.
5. Describe how the FIFO strategy differs from other Branch and Bound strategies like LIFO and Best-First Search.
6. Explain how the LIFO (stack-based) strategy is used in the Branch and Bound algorithm to explore the solution space.
7. How does the LIFO Branch and Bound differ from the FIFO and Best-First Branch and Bound approaches?
8. Explain how the Least Cost Branch and Bound technique determines which node to explore next in the search space using suitable examples.
9. Describe the difference between Least Cost Branch and Bound and the general Branch and Bound approach.
10. Explain the process of solving the 0/1 Knapsack problem using Branch and Bound. How are upper bounds used in this method?
11. How does the Branch and Bound technique for the 0/1 Knapsack problem ensure that the solution found is optimal?
12. Explain how the Branch and Bound method is used to solve the TSP. What role do lower bounds play in this method?

13. Describe the difference between solving TSP using Branch and Bound versus using brute force.

Moderate Questions (Bloom's Level: Applying)

1. Demonstrate how the FIFO Branch and Bound method can be applied to solve a simple 0/1 Knapsack problem.
2. Apply the FIFO Branch and Bound approach to solve a small Traveling Salesman Problem (TSP).
3. Apply the LIFO Branch and Bound approach to solve the 0/1 Knapsack problem. Show the steps involved.
4. Use LC Branch and Bound to find a solution to a Traveling Salesman Problem (TSP) with 4 cities.
5. Apply the Branch and Bound method to solve a 0/1 Knapsack problem with 3 items, given specific weights and values.
6. Use Branch and Bound to demonstrate how to discard suboptimal branches when solving a 0/1 Knapsack problem.
7. Apply the Least Cost Branch and Bound method to solve a small 0/1 Knapsack problem.
8. Demonstrate how to use the Least Cost Branch and Bound algorithm to solve a simple 5-city Traveling Salesman Problem (TSP).
9. Apply the Branch and Bound method to solve a small Traveling Salesperson Problem for 4 cities with a given distance matrix.
10. Use the Branch and Bound approach to demonstrate how to prune suboptimal paths when solving a simple TSP.
11. There are 5 bags labelled 1 to 5. All coins in a given bag have same weight. Some bags have coins of weight 10 gms, others have coins of weight 11 gms. I pick 1, 2, 4, 8, 16 coins from bags 1 to 5 respectively. Their total weight is 323 gms. What is the product of the labels of the bags having 11 gm coins.

Higher Order Thinking Skills (HOTS) Questions (Bloom's Level: Analyze)

1. Analyze the impact of using a FIFO queue on the efficiency of the Branch and Bound algorithm in terms of time and space complexity.
2. Compare the performance of FIFO Branch and Bound with Best-First Search Branch and Bound for solving optimization problems.
3. Analyze the strengths and weaknesses of using LIFO over FIFO in terms of time complexity and memory usage in the Branch and Bound algorithm.

4. Use LC Branch and Bound to find a solution to a Traveling Salesman Problem (TSP) of a given matrix.

∞ 10 5

8 ∞ 9

4 6 ∞

5. How does the LIFO approach affect the order in which nodes are explored? Provide an example to illustrate your answer.

6. Compare the effectiveness of the Least Cost Branch and Bound approach with the Best-First and Depth-First Branch and Bound methods in terms of memory and time complexity.

7. Analyze the time complexity of solving the 0/1 Knapsack problem using the Branch and Bound approach. How does it compare with Dynamic Programming?

8. Compare the Branch and Bound method for the 0/1 Knapsack problem with the Greedy method in terms of optimality and efficiency.

Higher Order Thinking Skills (HOTS) Questions (Bloom's Level: Evaluate)

1. Evaluate the situations where the FIFO Branch and Bound approach would be more efficient than other variations of the Branch and Bound algorithm.

2. Evaluate the scenarios in which the LIFO Branch and Bound technique is more efficient compared to other strategies like Best-First or FIFO.

3. Evaluate the advantages of using the Least Cost Branch and Bound method for solving optimization problems like the Traveling Salesman Problem (TSP).

4. Evaluate the effectiveness of different bounding functions (e.g., linear relaxation) used in Branch and Bound for solving the 0/1 Knapsack problem.

5. Propose an enhancement to the standard Branch and Bound algorithm for solving the 0/1 Knapsack problem to improve its performance on larger datasets.

Backtracking

CO4: Apply Backtracking approach to implement solutions for combinatorial problems.

Basic Questions (Bloom's Level: Understanding)

1. Explain the concept of backtracking in problem-solving. How does it differ from brute force approaches?
2. Apply backtracking approach to write a algorithm and diagonal logic to solve the 8-Queens problem.
3. Discuss the role of recursion in backtracking algorithms. Why is recursion essential in implementing backtracking strategies?
4. Differentiate between backtracking and dynamic programming in terms of problem-solving approach. Provide an example where backtracking would be more appropriate.
5. Explain how backtracking fits into the overall design paradigm of algorithms. How does it approach problem-solving differently from greedy or divide-and-conquer strategies?
6. Illustrate with an example how backtracking ensures correctness in finding all feasible solutions to an optimization problem.
7. Summarize the process of backtracking in solving the Hamiltonian Path problem. How does backtracking ensure that no path is missed?
8. Explain why backtracking is often referred to as a depth-first search (DFS) approach. How do they relate in terms of algorithm structure?
9. Differentiate between the types of problems that are best solved using backtracking versus those better suited for greedy algorithms. Provide an example for each.
10. Which of the following is NOT a characteristic of the backtracking algorithm?
    A. Recursive approach
    B. Breadth-first exploration
    C. Depth-first exploration
    D. Trial and error

Moderate Questions (Bloom's Level: Applying)

1. Demonstrate how the backtracking algorithm can be used to place 8 queens on a chessboard such that no two queens threaten each other. What specific conditions would you check at each step?

2. Using backtracking, show how you would assign colors to the vertices of a graph such that no two adjacent vertices share the same color. Apply this approach to a graph with 5 vertices and 3 colors.
3. Given a set of numbers and a target sum, demonstrate how backtracking can be used to find all subsets that sum up to the target. Use the set {1, 3, 9, 2, 7} and target sum of 10 to illustrate the process.
4. Using the backtracking approach, write the pseudocode to solve the 8-Queen problem. Then, manually place queens on a 4x4 chessboard and demonstrate how the algorithm proceeds step by step.
5. Given a partially completed chessboard where 4 queens have already been placed, use backtracking to place the remaining queens. Show the sequence of recursive calls and decisions.
6. Implement the backtracking solution for the 8-Queen problem in a programming language of your choice. Explain how you would handle situations where placing a queen in a specific position leads to no further valid moves.
7. Modify the backtracking algorithm for the 8-Queen problem to count the total number of valid solutions instead of finding just one solution. Demonstrate the changes in the algorithm and show an example output.
8. Apply the backtracking method to solve a variant of the 8-Queen problem where queens must be placed on a 10x10 chessboard. What modifications are required, and how does the search space change?
9. Using backtracking strategy to write the pseudo code to solve a graph coloring problem with 4 vertices and 3 colors. Apply the algorithm to a simple graph and demonstrate the step-by-step process.
10. Given a specific graph (e.g., a triangle graph), use backtracking to assign colors to each vertex such that no adjacent vertices share the same color. Illustrate the recursive process and show how conflicts are resolved.
11. Implement a backtracking algorithm to color a graph where each vertex must be colored with one of 4 colors. Apply this algorithm to a provided adjacency matrix and explain how you handle color assignments and backtracking decisions.
12. Modify the backtracking approach to solve the graph coloring problem with the minimum number of colors needed. Demonstrate your modified algorithm on a graph with 5 vertices and explain how you optimize the color count.

Higher Order Thinking Skills (HOTS) Questions (Bloom's Level: Analyze)

1. Analyze the time complexity of the backtracking algorithm used to solve the 8-Queen problem. How does the search space affect the overall performance of the algorithm?
2. Compare the backtracking approach for the 8-Queen problem with other potential approaches (e.g., brute force). Which approach is more efficient, and why?
3. Examine the process of placing queens in the 8-Queen problem. At what stages does the algorithm backtrack, and how does it decide which queen to reposition? Provide a detailed explanation of the decision-making process.
4. Analyze the efficiency of the backtracking algorithm for solving the graph coloring problem. What are the factors that influence the performance of the algorithm, and how can these factors be optimized?
5. Compare the backtracking approach for graph coloring with other techniques like greedy algorithms or dynamic programming. What are the strengths and weaknesses of each method in terms of time complexity and solution quality?
6. Examine the point at which backtracking occurs in the graph coloring problem. How does the algorithm decide when to backtrack, and how does this impact the search for a solution?
7. Using backtracking method, solve given 0/1 Knapsack problem s: Weights : (5, 3, 4, 2) Profit: 20, 15, 12, 10 where Knapsack capacity (M) = 7
8. Apply backtracking method, to get maximum profit using 0/1 Knapsack Space tree for given values: Weights : (2, 1, 3, 2) Profit: 12, 10, 20, 15 where Knapsack capacity (M) = 5 .